

个人财务记账系统项目文档

陈伟栋 (2312190218) 曾昭祥 (2312190219)

一、项目介绍

1.1 团队成员与分工

姓名	角色	主要负责
陈伟栋	后端开发	数据库设计与迁移、RESTful API、核心业务逻辑、后端测试、Docker/后端部署、安全审查、监控端点
曾昭祥	前端开发	UI/UX 设计与实现、前端架构、API 访问层、数据可视化、前端测试与 CI、Vercel 前端部署

1.2 背景与问题陈述

随着移动支付普及，大学生与年轻职场用户的日常消费高度依赖微信、支付宝等渠道，账单分散、记账成本高、月末复盘困难等问题日益突出。传统纸质记账或简单 Excel 表格难以满足「快速录入、分类统计、预算控制、智能分析」的综合需求。

本项目开发一款名为**个人财务记账系统**的响应式 Web 应用，帮助用户：

- 快速记录收入、支出与转账；
- 管理多账户资产；
- 设置预算并实时预警；
- 导入微信账单批量入账；
- 借助大模型获得消费分析与理财建议。

市面同类产品虽多，但仍存在以下痛点，本项目针对性解决：

痛点	本项目方案
账单分散、手工录入繁琐	支持微信账单批量导入 + 后端自动分类
分类依赖用户记忆	规则 + DeepSeek 三级分类策略
数据展示停留在数字罗列	ECharts 多维图表 + AI 文字报告
前后端协作与工程质量难保障	RESTful API 约定 + pytest/Vitest + GitHub Actions CI

1.3 项目目标与价值

- 功能目标：**实现可交互的完整财务管理系统，覆盖记账、账户、预算、统计、导入、AI 分析等核心链路。
- 技术目标：**掌握 Vue 3 + TypeScript 前端工程化、FastAPI 后端分层、MySQL/Redis 持久化、Docker 与云部署、大模型 API 集成。

- **学习价值：**实现前后端分离、API 契约驱动开发、自动化测试与 CI/CD、安全审查与可观测性配置。

1.4 功能需求

功能性需求

功能模块	描述
用户认证	注册、登录、JWT 鉴权、个人信息与密码管理
交易记账	收入/支出/转账三类交易记录，分类与账户联动，列表筛选
账户管理	多账户（现金/银行卡/微信/支付宝等），转账与余额调整
预算管理	月度总预算与分类子预算，使用率进度与超支预警
统计分析	收支趋势、分类占比、Excel 报表导出
微信导入	CSV/XLSX 解析、预览确认、去重、导入后 AI 自动分类
AI 智能分析	基于历史数据的理财建议、历史记录
交易记录分类	自定义分类树

非功能性需求

- **性能：**页面切换流畅，图表按需渲染；后端异步 I/O。
- **稳定性：**前后端统一异常处理；财务金额使用 `DECIMAL` 精确存储。
- **安全性：**BCrypt 密码哈希、JWT + Redis 黑名单、OWASP 审查与 CI 密钥扫描。

1.5 技术选型概述

层级	选型	理由
前端框架	Vue 3 + TypeScript + Vite	组合式 API、类型安全、快速 HMR
UI	Element Plus	成熟桌面端组件，表单/表格/步骤条齐全
状态/路由	Pinia + Vue Router 4	轻量状态管理 + 路由守卫
图表	ECharts 5	仪表盘、统计页、AI 预算图
后端	FastAPI + SQLAlchemy 2.0	自动文档、依赖注入、异步友好
数据库	MySQL 8.0 + Redis 7	关系型主库 + Token 黑名单/缓存
AI	DeepSeek Chat	国内可访问、OpenAI SDK 兼容
部署	Docker Compose + Vercel + Railway	本地四容器；线上前后端分离

说明： `package.json` 中安装了 Vant，但实际代码**未引用**，实际 UI 全部为 Element Plus。

1.6 在线访问

环境	地址
前端 (Vercel)	https://personal-financial-management-assis.vercel.app
后端 API (Railway)	https://personal-financial-management-assistant-production.up.railway.app/api

二、需求分析与 UI/UX 设计

2.1 用户画像与典型场景

目标用户：大学生、初入职场的年轻人、高频使用微信/支付宝的数字支付用户。

典型场景：

- 1. **快速记账：**方便完成收入支出的录入。
- 2. **月初导入微信账单：**上传微信账单 → 预览 → 确认；后端对未指定分类的记录自动 AI 分类。
- 3. **预算预警：**设置分类预算后实时查看使用率，接近或超过阈值时视觉预警。
- 4. **月度复盘与 AI 建议：**查看趋势/占比图表，在「智能分析」页生成个性化报告。

2.2 界面原型设计：v0.dev

本项目的 UI/UX 设计主要通过v0.dev实现：

阶段	工具	作用
初版高保真原型	v0.dev	根据功能描述快速生成核心页面效果图，统一风格（主色 #16A34A），作为开发的视觉基准，确定跳转逻辑与交互效果
设计归档与展示	Figma	将 v0 导出的 PNG 导入 Figma 项目，便于课程提交与查看

Figma 链接： [Finance System Design Prototype](#)

初版设计效果图：

(1) 仪表盘 Dashboard

仪表盘

交易记录

记一笔

统计分析

预算管理

账户管理

导入账单

财务仪表盘



本月收入

¥190.2

↑ 93.89%



本月支出

¥1,835.78

↑ 34.25%



本月结余

¥-1,645.58

2025年12月



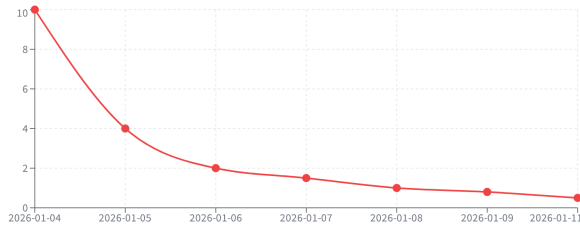
总资产

¥1,172.98

所有账户资金

最近7天支出趋势

● 支出



支出分类占比



餐饮	35%
其他支出	22%
娱乐	18%
学习	12%
交通	8%
抵账	5%

快速操作

+记一笔

记查看统计

预算管理

导入账单

(2) 记账 Add Transaction

- 仪表盘
- 交易记录
- 记一笔
- 统计分析
- 预算管理
- 账户管理
- 导入账单

快速记账

支出

收入

转账

金额

¥ 0.00

分类



购物



早餐



餐饮



交通



公交



午餐



娱乐



打车



家装



加油



买菜



购物



停车



学习



零食



医疗



饮料



高速



住房



养娃



通讯



其他支出



余额理财

账户

test (余额 ¥1072.98)

交易时间

2026-01-10 16:35:20

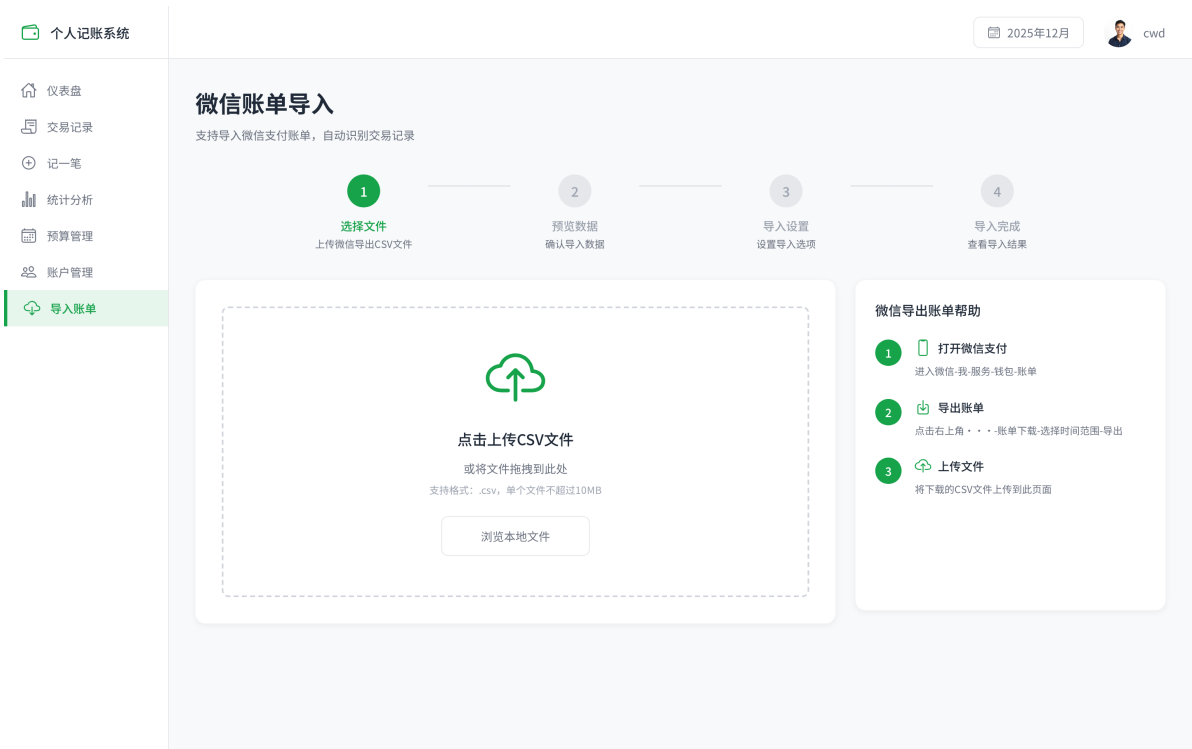
备注

添加备注...

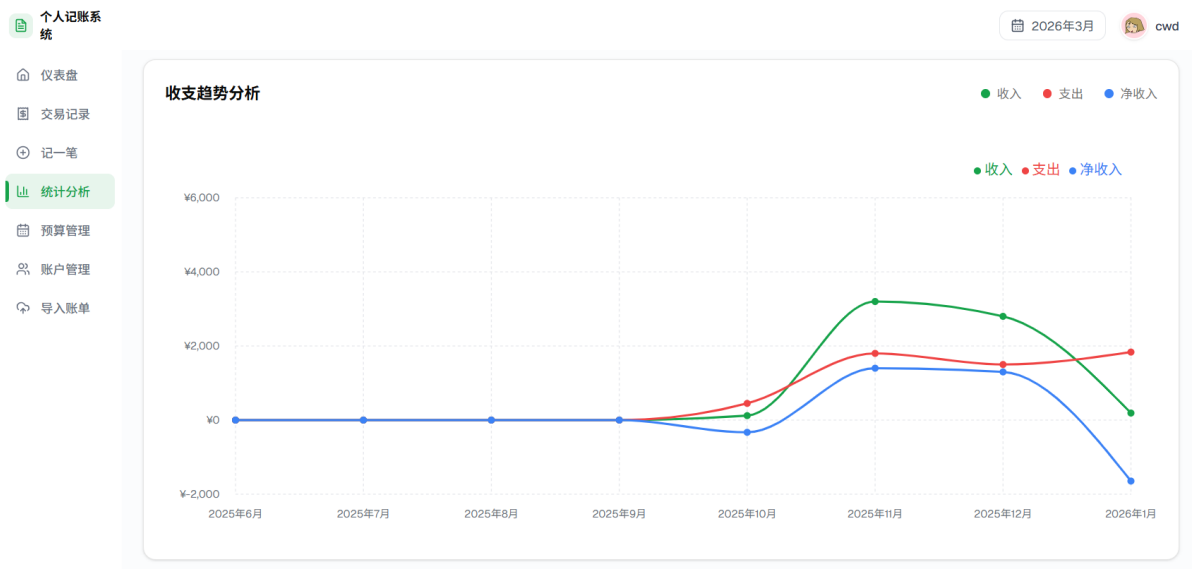
0 / 200

确认支出

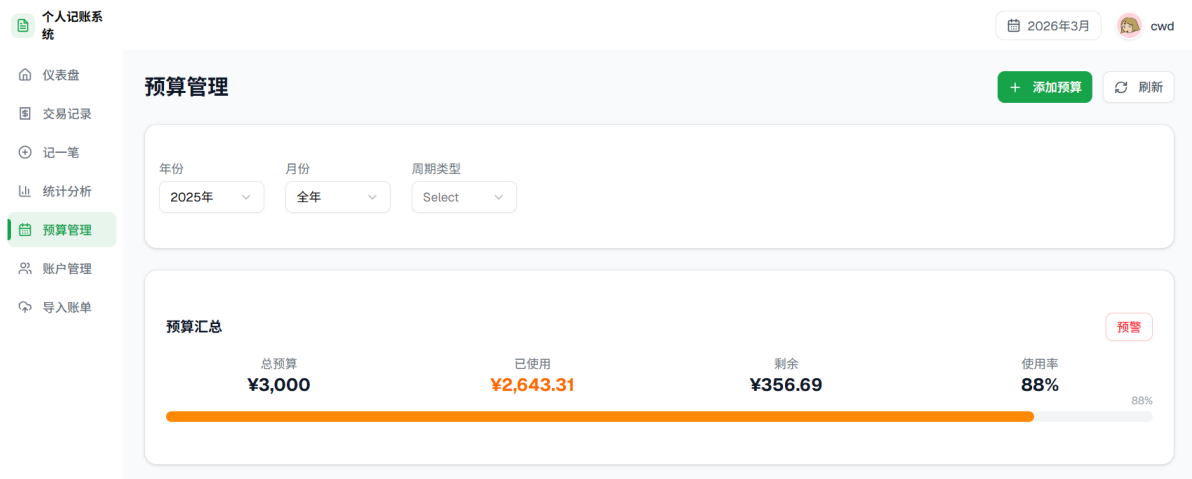
(3) 微信账单导入 Wechat Import



(4) 统计分析 Statistics



(5) 预算管理 Budget



最终实现与初版设计的对应关系：开发阶段以 v0 效果图为准，使用 Element Plus 组件在 Vue 中复现布局与配色；部分细节与后续迭代新增功能会有所不同（如 AI 智能分析页、侧边栏「智能分析」入口等）。

2.3 设计规范

统一视觉规范写入 docs/design-spec.md，核心 Token 如下：

角色	色值	用途
主色 Primary	#16A34A	主按钮、正向数据
辅助色 Secondary	#0EA5E9	链接、图表辅助色
强调色 Accent	#F59E0B	预算接近阈值
危险色 Error	#EF4444	超支、删除、错误

字体：中文 PingFang SC / Microsoft YaHei；正文 14px，标题 18-24px；间距采用 4px 栅格。

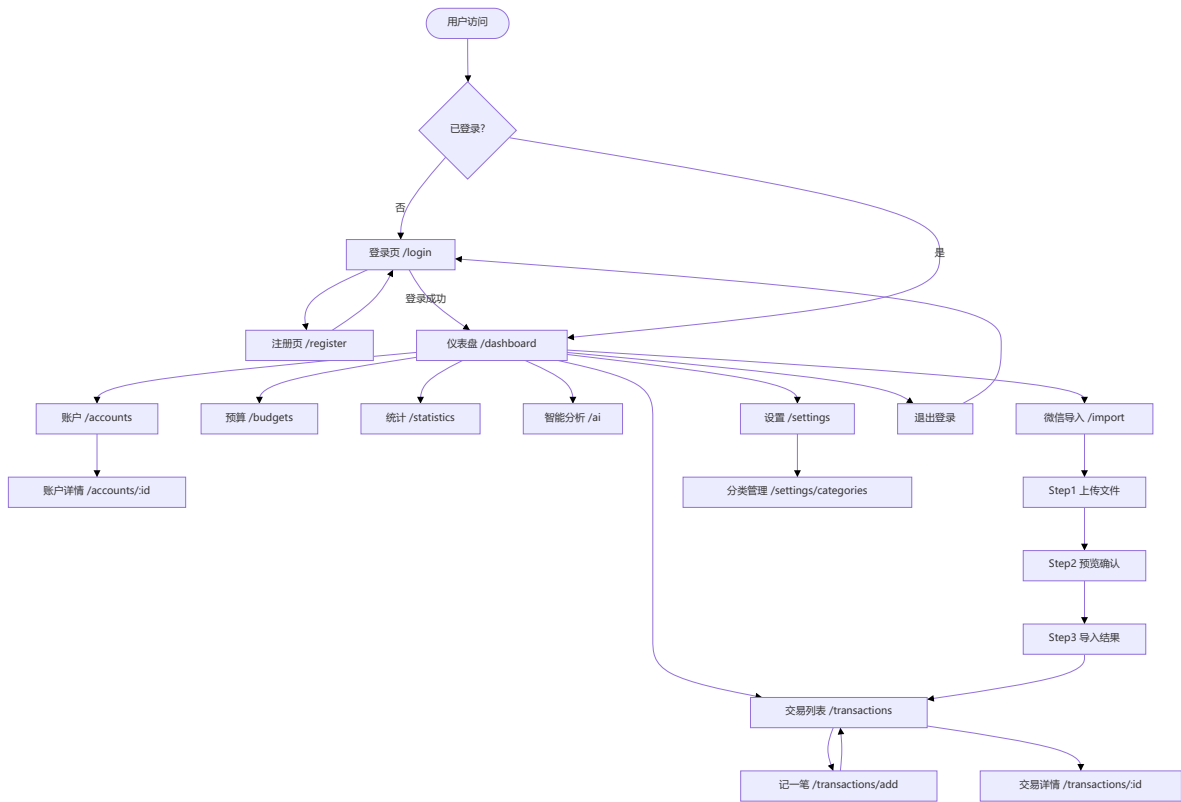
2.4 信息架构

侧边栏主导航（AppSidebar.vue）与路由一一对应：

导航项	路由	页面组件
仪表盘	/dashboard	DashboardView.vue
交易记录	/transactions	TransactionList.vue
账户管理	/accounts	AccountList.vue
预算管理	/budgets	BudgetView.vue
统计分析	/statistics	StatisticsView.vue
智能分析	/ai	AIView.vue
数据导入	/import	wechatImport.vue
个人设置	/settings	ProfileView.vue

Guest 路由：/login、/register、/forgot-password。

2.5 界面跳转流程图



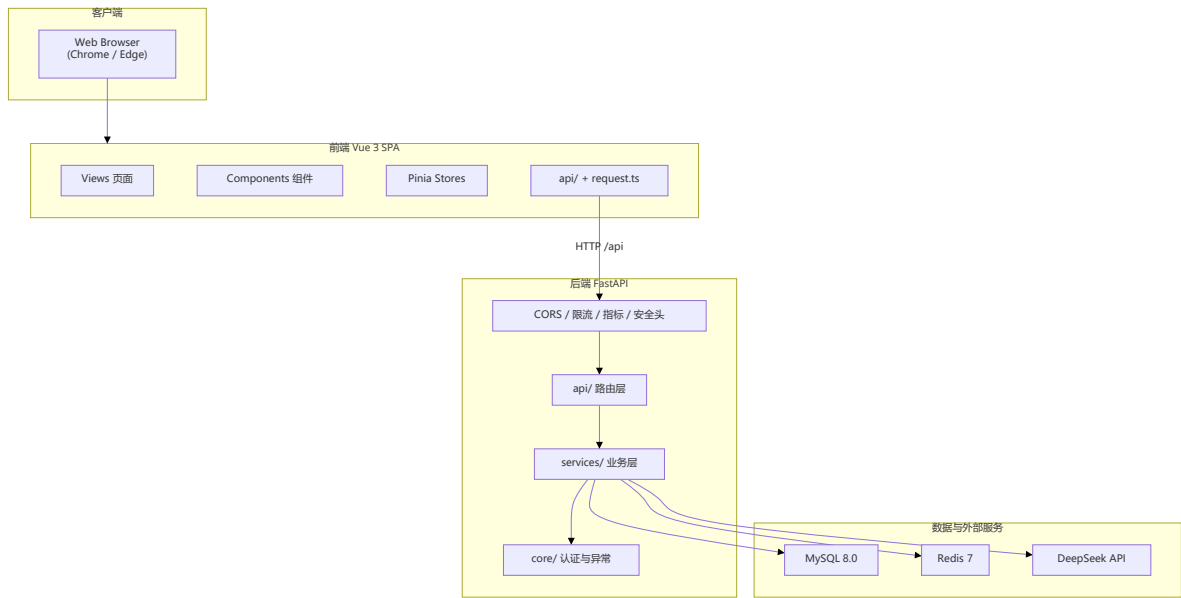
- 说明：
- 受保护页面均挂载在 `MainLayout` 下，`meta.requiresAuth: true`。
 - 路由守卫在 `router/index.ts` 中读取 `TokenManager.getAccessToken()`，无 Token 则重定向 `/login`。
 - 微信导入采用 `el-steps` 三步骤条，由组件内 `activeStep` 控制，非独立路由。

三、系统架构设计

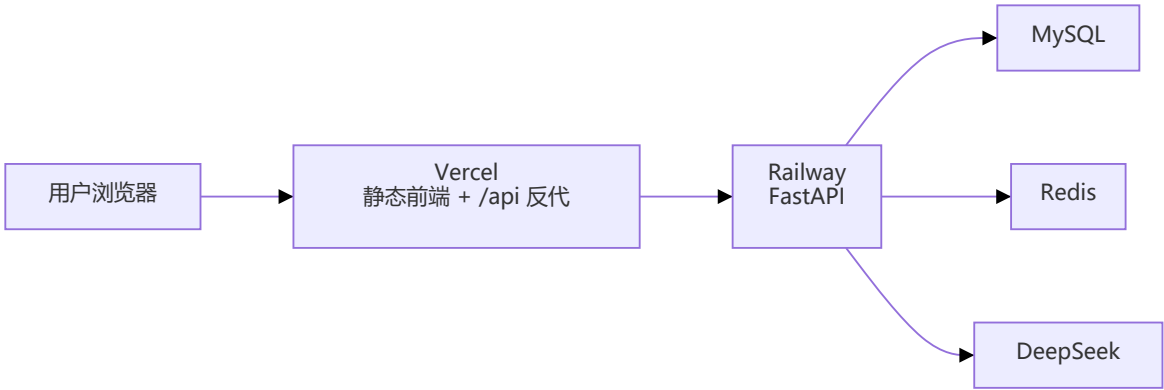
3.1 整体架构图

系统采用前后端分离 + RESTful API 架构，本地开发与生产部署结构如下。

系统架构图：



线上部署架构：



3.2 前端架构分层

层级	目录	职责
视图层	views/	页面与业务 UI
组件层	components/	可复用 UI 组件
状态层	stores/	Pinia 全局状态
API 层	api/	REST 接口封装
基础设施	request.ts 等	拦截器、Token、路由

路由守卫实现（鉴权核心逻辑）：

```
// frontend/src/router/index.ts
router.beforeEach((to, from, next) => {
  const token = TokenManager.getAccessToken()
  const requiresAuth = to.matched.some(record => record.meta.requiresAuth)
  const isGuestRoute = to.matched.some(record => record.meta.guest)

  if (requiresAuth && !token) {
    next('/login')
  } else if (isGuestRoute && token) {
    next('/')
  } else {
    next()
  }
})
})
```

HTTP 请求封装（统一 Base URL 与 Token 注入）：

```
// frontend/src/api/request.ts
const service = axios.create({
  baseURL: import.meta.env.VITE_API_BASE_URL || '/api',
  timeout: 10000,
  headers: { 'Content-Type': 'application/json' }
})

service.interceptors.request.use((config) => {
```

```

const token = TokenManager.getAccessToken()
if (token) {
  config.headers.Authorization = `Bearer ${token}`
}
return config
})

service.interceptors.response.use((response) => {
  const res = response.data
  if (res.code && res.code !== 200) {
    ElMessage.error(res.message || 'Error')
    return Promise.reject(new Error(res.message || 'Error'))
  }
  return res.data
})

```

说明：

前端采用**单向数据流 + 分层调用**模式：页面组件不直接拼 URL，而是调用 `src/api/*.ts` 中的函数；需要跨页面共享的状态（登录用户、AI 建议）放入 Pinia Store；纯工具逻辑（Token 读写）放在 `utils/`。

(1) 视图层 `views/`

每个业务模块对应一个子目录（如 `transaction/`、`import/`），页面负责布局、表单绑定、Loading/Empty 态展示。复杂图表与卡片拆到 `components/business/`，避免单文件过长。

(2) API 层 `api/` + `request.ts`

`request.ts` 创建全局 Axios 实例，统一 `baseUrl`、超时与拦截器。各模块文件（`auth.ts`、`transactions.ts` 等）只需关注路径与参数，返回类型与 `src/types/` 对齐。响应拦截器解包 `{ code, message, data }` 后直接返回 `data`，页面代码写 `const list = await getTransactions()` 即可。

(3) 状态层 `stores/`

`user.ts` 管理登录态：登录成功后写 Token 并拉 `/auth/me`；登出时清 Store 与 `localStorage`。
`ai.ts` 管理理财建议的 loading、当前建议、历史分页，避免在 `AIView.vue` 内堆叠过多异步逻辑。

(4) 路由与鉴权

受保护路由挂在 `MainLayout` 下并设 `meta.requiresAuth: true`；`beforeEach` 守卫读取 `localStorage` 中的 Token，未登录访问业务页会重定向 `/login`，已登录用户访问 `/login` 会被送回首页。

(5) 环境与代理

- 开发：`vite.config.ts` 将 `/api` 代理到 `http://127.0.0.1:8000`（避免 Windows 下 `localhost` 走 IPv6 导致连不上后端）。
- Docker/Nginx：浏览器请求同源 `/api`，由 Nginx 反代 `backend:8000`。
- Vercel：`vercel.json` 的 `rewrites` 将 `/api/*` 转发到 Railway，前端构建时 `VITE_API_BASE_URL` 设为 `/api` 即可。

3.3 后端架构分层

层级	目录	职责
入口	<code>main.py</code>	应用创建、中间件、生命周期
路由层	<code>app/api/</code>	HTTP 端点，参数校验，依赖注入
业务层	<code>app/services/</code>	核心业务逻辑
模型层	<code>app/models/</code>	SQLAlchemy ORM
校验层	<code>app/schemas/</code>	Pydantic v2 请求/响应模型
核心层	<code>app/core/</code>	JWT、依赖、异常、限流、指标

应用入口与中间件（生产日志 + 安全头 + 指标采集）：

```
# backend/main.py
setup_logging(
    debug=settings.debug,
    json_output=(settings.app_env == "production"),
)

app.add_middleware(CORSMiddleware, ...)
app.add_middleware(RateLimitMiddleware)
app.add_middleware(MetricsMiddleware)
app.add_middleware(SecurityHeadersMiddleware)

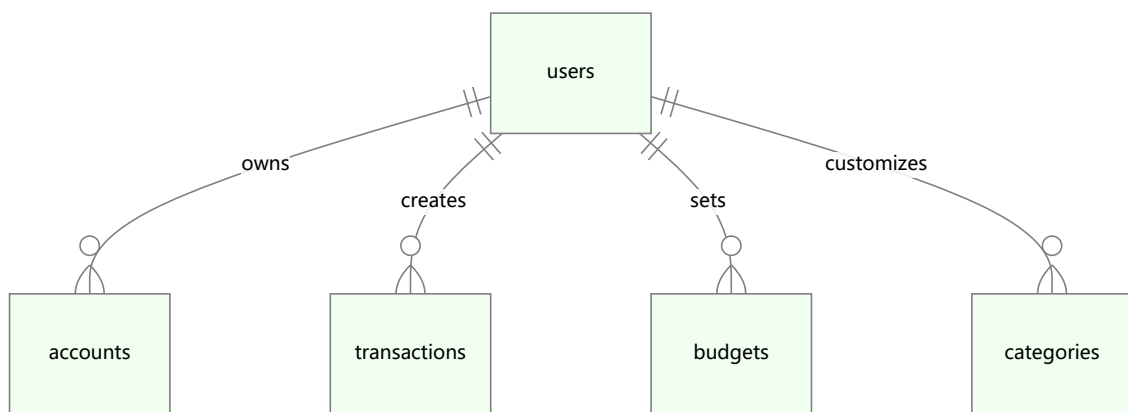
app.include_router(api_router, prefix="/api")
```

统一响应格式（前后端约定）：

```
{
  "code": 200,
  "message": "success",
  "data": { }
}
```

3.4 数据库设计

共 9 张核心表，详见 `docs/database.md`。ER 关系如下：



设计要点:

(1) 用户与数据隔离

所有业务表均含 `user_id` 外键; Service 层查询统一过滤 `user_id == current_user.id`, 防止通过修改 URL 中的 ID 访问他人数据。

(2) 金额精度

交易、账户余额、预算等字段使用 `DECIMAL(15,2)` 存储, 避免浮点误差。FastAPI 序列化后常以**字符串**下发 (如 `"1000.00"`), 前端展示须 `Number(value || 0).toFixed(2)`, 不可直接对字符串调用 `.toFixed()` (曾导致账户/预算页无数据)。

(3) 分类体系

`categories` 支持系统预置分类与用户自定义; 树形结构通过 `parent_id` 自关联。预算、交易、提醒均可关联 `category_id`。

(4) 交易与账户联动

创建/更新/删除交易时, `TransactionService` 同步调整 `accounts.balance` 并写入 `balance_history`, 保证账本与账户余额一致。转账类型需同时维护转出/转入账户。

(5) 微信导入与去重

导入交易 `source='wechat'`, 可存 `wechat_transaction_id`。预览与入库阶段按微信单号或「时间+金额+商户」组合检测重复, 跳过已存在记录。

(6) AI 建议缓存

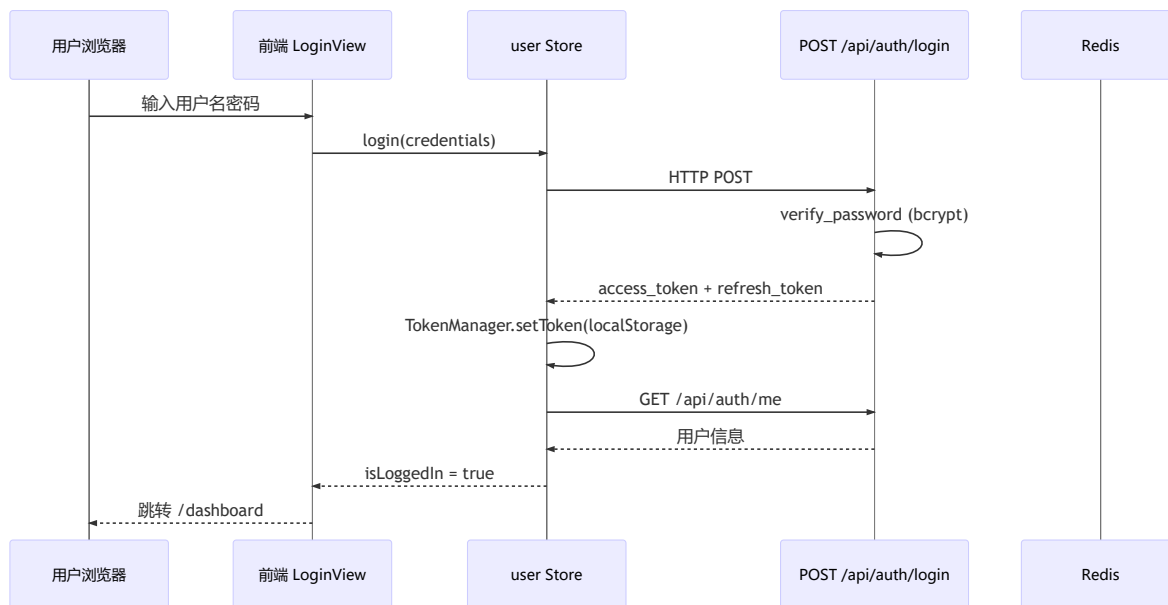
`ai_advice_records` 表存储历史理财建议 JSON, 相同分析周期可命中缓存直接使用, 减少 LLM 调用; `force_refresh=true` 时强制重新生成。

(7) 迁移与版本

表结构变更通过 Alembic 脚本管理 (`backend/alembic/versions/`), 禁止在生产环境手工 DDL; CI 测试使用 SQLite, 与 MySQL 字段类型在 Schema 层做兼容处理。

3.5 系统核心业务流程

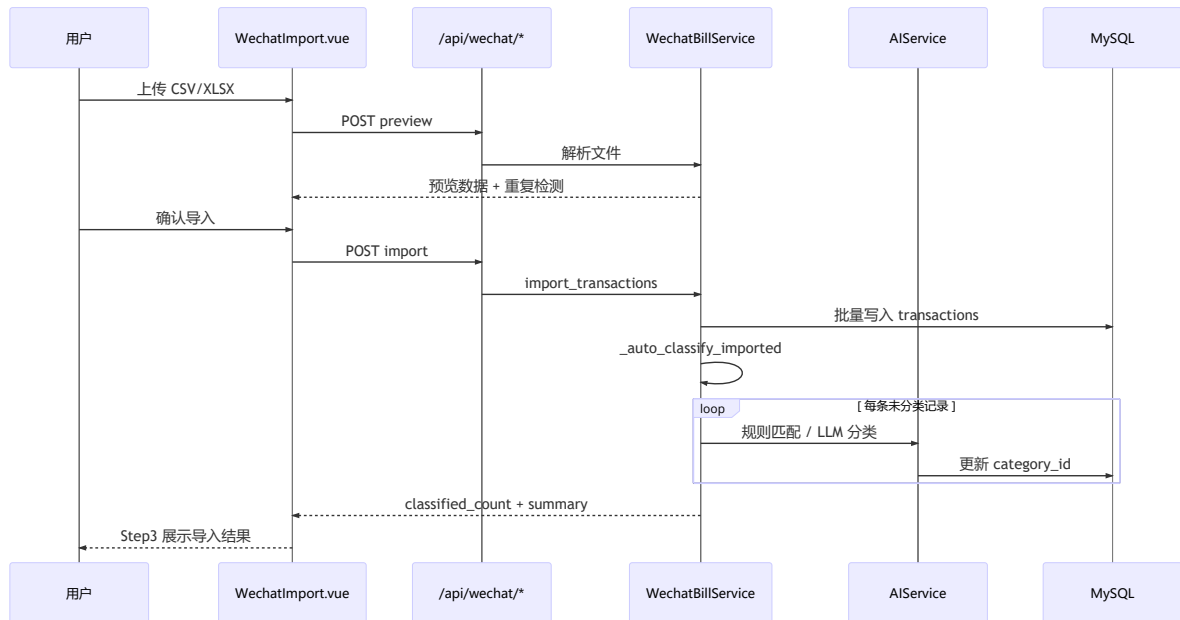
3.5.1 用户登录流程



后端登录核心代码：

```
# backend/app/api/auth.py
access_token = create_access_token(
    data={"sub": str(user.id)}, # sub 必须为字符串
    expires_delta=timedelta(minutes=settings.access_token_expire_minutes)
)
return Response(code=200, message="登录成功", data={
    "access_token": access_token,
    "refresh_token": refresh_token,
    "token_type": "bearer",
})
```

3.5.2 微信账单导入 + AI 自动分类



导入后自动分类代码：

```

# backend/app/services/wechat_bill_service.py
if auto_classify and imported_transactions:
    classified_count = self._auto_classify_imported(
        db, user_id, imported_transactions
    )
if classified_count > 0:
    summary += f", AI分类{classified_count}条"
  
```

前端三步向导（WechatImport.vue 使用 el-steps + activeStep）：

```
<el-steps :active="activeStep" finish-status="success" align-center>
  <el-step title="上传文件" />
  <el-step title="预览确认" />
  <el-step title="导入结果" />
</el-steps>
```

微信账单导入

支持微信支付导出的 CSV / XLSX 格式账单文件

导入历史

导出说明

1 上传文件

2 预览确认

3 导入结果



拖拽账单文件到此处，或 [点击选择](#)

支持微信支付导出的 .csv 和 .xlsx 文件，最大 10 MB

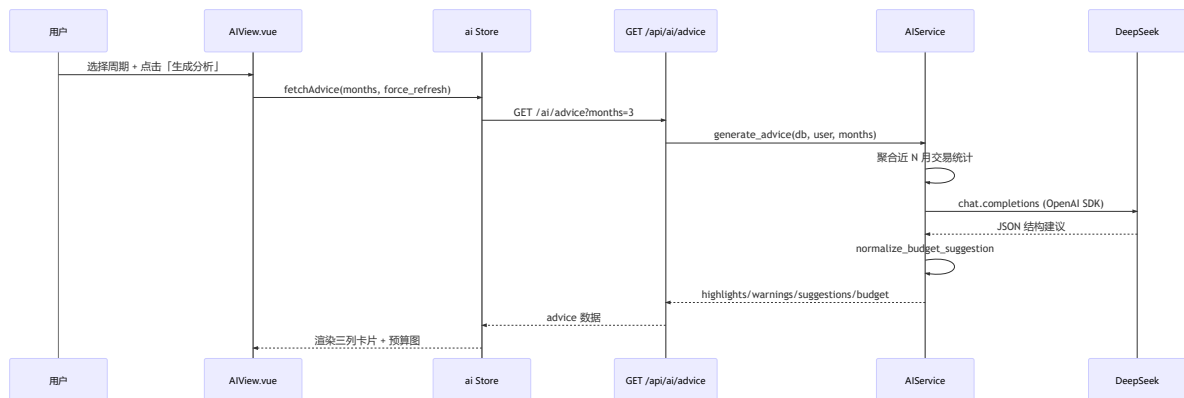
如何导出微信账单？

1 打开微信 → 我 → 服务 → 钱包

2 点击右上角 账单 → 常见问题 → 下载账单

3 选择时间范围后发送至邮箱，解压压缩包后上传 CSV 或 XLSX 文件

3.5.3 AI 理财建议生成



前端 AI 页核心结构：

```

<!-- AView.vue: 三列建议 + 预算图 -->
<AIAdviceCard title="消费亮点" type="highlight" :items="..." />
<AIAdviceCard title="优化建议" type="suggestion" :items="..." />
<AIAdviceCard title="风险提醒" type="warning" :items="..." />
<AIBudgetChart :breakdown="budgetBreakdown" :total="totalBudget" />

```




四、版本控制与协作开发

项目采用 **GitHub Flow** 变体: `main` (稳定) ← `dev` (集成) ← `feature/*` (功能分支)。

分支	用途
<code>main</code>	生产稳定代码, Vercel/Railway 自动部署源
<code>dev</code>	开发集成分支
<code>feature/曾昭祥-frontend-doc</code>	前端开发
<code>feature/cwd.backend</code>	后端开发

功能完成后提交 PR 合并至 `dev`，再 PR 合并至 `main`。

五、前端应用实现

5.1 工程化配置

- **Vite 5**: 路径别名 `@` → `src/`，开发代理 `/api` → 后端。
- **TypeScript 5.3**: 全项目类型约束, `src/types/` 与 API 一一对应。
- **ESLint + Prettier**: `npm run lint` 零警告门禁 (CI 同样执行)。

5.2 核心页面实现说明

5.2.1 认证模块

- `LoginView.vue` / `RegisterView.vue`: Element Plus 表单校验。
- 登录调用 `useUserStore().login()` → 写 Token → 拉 `/auth/me` → 跳转仪表盘。

```
// stores/user.ts
const login = async (loginData: UserLogin) => {
  const response = await loginApi(loginData)
  TokenManager.setToken({ access_token, refresh_token, token_type: 'bearer' })
  await getUserInfo()
}
```

5.2.2 仪表盘

- 四指标卡片：本月收入、支出、结余、总资产。
- `TrendChart` 折线图 + `CategoryPieChart` 饼图。
- 数据来源： `/api/transactions/summary`、 `/api/statistics/*`。

5.2.3 交易模块

- **列表** `TransactionList.vue`：时间筛选、分类筛选、分页； `TransactionCard` 展示单笔；未分类可弹窗手动选分类。
- **新增** `TransactionAdd.vue`：支出/收入/转账 Tab + `AmountInput` + 分类 Grid。
- **详情** `TransactionDetail.vue`：编辑、删除。

5.2.4 账户与预算

- `AccountList.vue`：卡片式账户、资产占比、跳转详情与转账。
- `BudgetView.vue`： `BudgetProgress` 进度条，超支变红/橙预警。

5.2.5 统计分析

- 日期范围选择、收支趋势、分类占比、对比柱状图（ECharts）。

5.2.6 微信导入

- 三步 Stepper（见 3.5.2）； `FileUploader` / `el-upload` 拖拽上传。
- 导入设置可选默认账户、默认分类；结果页展示成功/跳过/失败/AI 分类条数。

5.2.7 AI 智能分析

- Tab：「智能建议」+「历史记录」。
- 周期选择 1/3/6/12 月； `force_refresh` 强制重新生成。
- 组件： `AIAdviceCard`、 `AIBudgetChart`；Store 管理 loading 与缓存标签。

5.3 可复用组件

组件	文件	说明
TransactionCard	<code>business/TransactionCard.vue</code>	列表卡片, Vitest 覆盖 13 用例
AmountInput	<code>business/AmountInput.vue</code>	财务金额输入
BudgetProgress	<code>business/BudgetProgress.vue</code>	预算使用率
TrendChart	<code>business/TrendChart.vue</code>	折线趋势
CategoryPieChart	<code>business/CategoryPieChart.vue</code>	分类饼图
FileUploader	<code>business/FileUploader.vue</code>	文件上传
MainLayout	<code>layout/MainLayout.vue</code>	侧边栏 + 顶栏布局

5.4 前后端联调要点

前后端分离开发中，**接口约定与运行时环境差异**是联调阶段最常见的问题来源。本项目在联调过程中重点处理了以下情况：

(1) 统一响应格式与 Axios 解包

后端所有成功响应为 `{ code: 200, message, data }`。前端在 `request.ts` 响应拦截器中校验 `code`，成功时直接 `return res.data`，业务层拿到的就是 `data` 字段内容。若后端返回 `code !== 200`，拦截器弹出 `ElMessage` 并 `reject`，页面 `catch` 即可处理失败态。

(2) JWT 与 401 处理

登录成功后 `TokenManager` 将 `access_token` 写入 `localStorage`；请求拦截器自动附加 `Authorization: Bearer ...`。Token 过期或无效时后端返回 401，响应拦截器清除 Token 并跳转 `/login`（已在登录页则不重复跳转），避免页面卡在半登录态。

(3) JWT sub 字段类型

`python-jose` 要求 JWT 的 `sub` 为字符串。后端创建 Token 时使用 `{"sub": str(user.id)}`，解码后再 `int(payload["sub"])` 查用户。若误传整数，会出现「Token 签发成功但 `/auth/me` 永远 401」的隐蔽问题。

(4) 金额 Decimal 序列化

MySQL `DECIMAL` 字段经 `Pydantic/FastAPI` 序列化后常为字符串（如 `"128.50"`）。Vue 模板若写 `{{ row.amount.toFixed(2) }}` 会抛 `toFixed is not a function` 导致白屏。统一改为 `Number(row.amount || 0).toFixed(2)` 或封装格式化函数。

(5) Windows 开发代理 IPv6

Vite 代理 `target: 'http://localhost:8000'` 时，Node 可能解析到 `:::1`，而后端 `uvicorn` 绑定 `127.0.0.1`，导致代理 `ECONNREFUSED`、前端 500。改为 `http://127.0.0.1:8000` 后恢复。

(6) 模块导出名不一致

曾出现 Store 导入 `getUserInfo` 但 API 导出 `getCurrentUser`，浏览器报 ES Module 语法错误、整页空白。联调时需保证 `import` 名称与 `export` 完全一致。

(7) 跨域与部署路径

本地 dev 靠 Vite proxy；Docker 靠 Nginx `/api` 反代；Vercel 靠 `rewrites`。三套环境均保持浏览器侧 `baseUrl: '/api'`，业务代码无需因部署环境改写 URL。

问题	现象	解决方案
Decimal 字符串	账户/预算页白屏	<code>Number(val).toFixed(2)</code>
localhost 代理	全部 API 500	代理 target 改为 <code>127.0.0.1</code>
JWT sub 类型	登录后 /me 401	后端 <code>str(user.id)</code>
401 未处理	旧 Token 假登录态	拦截器清 Token 跳登录
导出名错误	页面空白	对齐 API 与 Store 的 export 名

六、后端服务设计与实现

6.1 项目结构与依赖管理

backend/	
├─ main.py	# FastAPI 入口
├─ app/	
│ └─ api/	# 路由: auth, transactions, accounts, ai, wechat_bill...
│ └─ services/	# 业务逻辑
│ └─ models/	# ORM 模型
│ └─ schemas/	# Pydantic Schema
│ └─ core/	# security, dependencies, exceptions
│ └─ utils/	# logger, excel, date
├─ tests/	# pytest 测试
├─ alembic/	# 数据库迁移
└─ pyproject.toml	# uv 依赖管理

统一使用 `uv` 管理虚拟环境与依赖（`uv sync`、`uv run`）。

6.2 核心 API 模块

模块	前缀	主要能力
认证	<code>/api/auth</code>	注册、登录、刷新 Token、登出、/me、改密
交易	<code>/api/transactions</code>	CRUD、转账、汇总、分页筛选
账户	<code>/api/accounts</code>	CRUD、转账、余额调整、余额历史
分类	<code>/api/categories</code>	树形分类、系统默认 + 用户自定义
预算	<code>/api/budgets</code>	月度预算、分类子预算、执行监控
统计	<code>/api/statistics</code>	趋势、占比、对比、Excel 导出
微信	<code>/api/wechat</code>	预览、导入、历史、模板说明
AI	<code>/api/ai</code>	理财建议、历史（无 classify 对外接口）
健康	<code>/api/health</code>	存活 + MySQL/Redis 检查
指标	<code>/api/metrics</code>	请求数、错误率、响应时间

完整接口说明见文档 `docs/api.md` 与 `docs/api.yaml`。

6.3 认证与安全实现

- **密码**: `passlib` + BCrypt 哈希, 禁止明文存储。
- **JWT**: Access Token 30 分钟, Refresh Token 7 天; `sub` 字段为字符串类型用户 ID。
- **登出**: Access/Refresh Token 写入 Redis 黑名单。
- **鉴权依赖**: `get_current_active_user` 注入到受保护路由。
- **越权防护**: 所有查询带 `user_id` 过滤, 用户只能操作自己的数据。

6.4 微信账单服务

`WechatBillService` 负责:

1. 解析微信导出 CSV/XLSX (编码检测、字段映射) ;
2. 预览阶段重复检测 (`wechat_transaction_id` / 金额+时间) ;
3. 批量写入 `transactions`, 更新账户余额;
4. 对未指定 `category_id` 的记录调用 `_auto_classify_imported`。

6.5 AI 服务 (DeepSeek)

`AIService` (`app/services/ai_service.py`) 实现:

- (1) **规则分类器** `RULES`: 按商户关键词匹配餐饮/交通/购物等, 零 Token 消耗。

```
class AIService:
    """AI 智能服务类"""

    # 规则分类器 - 优先使用, 节省 Token
    RULES = {
        "餐饮": [
            "餐厅", "外卖", "咖啡", "奶茶", "火锅", "烧烤",
            "麦当劳", "肯德基", "瑞幸", "星巴克", "美团", "饿了么",
            "喜茶", "奈雪", "茶百道", "古茗", "蜜雪冰城",
            "肉包", "包子", "早阳", "早餐", "食堂", "饭店", "小吃",
        ],
        "交通": [
            "滴滴", "出租", "地铁", "公交", "加油", "停车",
            "高铁", "机票", "火车", "客车", "哈啰", "摩拜",
            "共享单车", "优步", "神州租车"
        ],
        "购物": [
            "淘宝", "京东", "拼多多", "超市", "便利店", "商场",
            "天猫", "苏宁", "国美", "唯品会", "当当", "亚马逊"
        ],
        "娱乐": [
            "电影", "游戏", "KTV", "酒吧", "旅游", "网吧",
            "影院", "乐园", "健身", "音乐会", "展览"
        ],
        "医疗": [
            "医院", "药店", "诊所", "体检", "药房", "挂号",
            "疫苗", "体检中心"
        ],
        "教育": [
            "培训", "课程", "书籍", "学费", "教育", "学习",
            "辅导", "考试", "学校", "大学", "一卡通", "校园",
        ],
        "住房": [
            "房租", "物业", "水电", "燃气", "采暖", "宽带",
            "装修", "家居", "房产"
        ]
    }
```

```
class AIService:
    ],
    "住房": [
        "房租", "物业", "水电", "燃气", "采暖", "宽带",
        "装修", "家居", "房产"
    ],
    "通讯": [
        "话费", "流量", "宽带", "充值", "通讯", "移动",
        "联通", "电信"
    ],
    "金融": [
        "理财", "基金", "股票", "保险", "银行", "借贷",
        "还款", "贷款", "证券"
    ],
}

# 默认分类（当无法匹配时使用）
DEFAULT_CATEGORY = "其他"

def __init__(self):
    """初始化 AI 服务"""
    self._client = None
```

- (2) **LLM 分类**：规则未命中时，构造 Prompt 调用 DeepSeek，返回分类 ID。
- (3) **理财建议** `generate_advice`：聚合用户近 N 月统计 → Prompt → 解析 JSON → `normalize_budget_suggestion` 归一化预算结构 → 可选写入 `ai_advice_records` 缓存。

环境变量（`.env.example`）：

```
AI_API_KEY=api_key
AI_BASE_URL=https://api.deepseek.com
AI_MODEL=deepseek-chat
```

七、AI 功能集成

7.1 功能清单

功能	触发方式	前端入口	后端实现
微信导入自动分类	导入确认后自动	导入结果页展示条数	<code>wechatBillService._auto_classify_imported</code>
个性化理财建议	用户点击「生成分析」	<code>/ai</code> 智能分析页	<code>GET /api/ai/advice</code>
建议历史	用户切换 Tab	AIView 历史 Tab	<code>GET /api/ai/advice/history</code>

7.2 提示词设计

两项 AI 能力均在后端 `AIService` 内构造 Prompt，经 DeepSeek Chat Completions API 调用（OpenAI SDK 兼容）。API Key 通过环境变量注入，不入库。

7.2.1 账单分类 Prompt（导入后内部调用）

当规则匹配 `_match_by_rules` 未命中时，将待分类交易列表与用户分类表拼入 Prompt，要求模型返回 JSON 数组（含 `index`、`category_id`、`confidence`）。核心结构如下：

```
prompt = f"""你是一个专业的账单分类助手。请根据以下信息，将每笔交易归类到最合适的分类中。

可用分类列表：
{category_text}

待分类的交易：
{items_text}

请以 JSON 格式返回结果，格式如下：
[
  {{
    "index": 0,
    "category_id": 123,
    "category_name": "餐饮",
    "confidence": 0.95
  }}
]
```

注意：

1. `index` 对应待分类交易的索引（从 0 开始）
2. `category_id` 必须从可用分类列表中选择
3. `confidence` 表示分类置信度（0-1 之间的浮点数）
4. 确保返回的是有效的 JSON 格式
5. 如果无法确定分类，请选择“其他”或最接近的分类

7.2.2 理财建议 Prompt（用户触发生成）

`generate_advice` 聚合近 N 月 `total_expense`、分类 breakdown 等统计，构造理财顾问角色 prompt，要求返回 `highlights`、`warnings`、`suggestions`、`next_month_budget` 等 JSON 字段：


```
prompt = f"""你是一位专业的理财顾问。请分析以下用户的消费数据，提供个性化的理财建议。

用户基本信息：
- 分析周期：最近 {months} 个月
- 总支出：{summary_data['total_expense']} 元
- 交易笔数：{summary_data['transaction_count']} 笔

分类支出明细：
{json.dumps(summary_data['category_breakdown'], ensure_ascii=False, indent=2)}

请以 JSON 格式返回分析结果，包含以下字段：
{{
  "highlights": ["消费亮点1", "消费亮点2"],
  "warnings": ["预警信息1", "预警信息2"],
  "suggestions": ["建议1", "建议2", "建议3"],
  "next_month_budget": {{
    "total": 金额数字,
    "breakdown": [
      {{ "category": "分类名", "suggested_amount": 金额数字 }}
    ]
  }},
  "full_report": "完整的理财建议报告文本"
}}
```

注意：

1. highlights 应该肯定用户的良好消费习惯
2. warnings 应该指出可能的消费问题
3. suggestions 应该提供具体的改进建议
4. next_month_budget 根据历史数据给出合理的下月预算建议
5. full_report 是一段完整的、友好的理财建议文本
6. 确保返回的是有效的 JSON 格式

```
"""
```

7.3 分类策略详解（见6.5）



八、软件测试

8.1 前端测试（Vitest）

类型与数量：

类型	文件	用例数	说明
单元测试	auth.test.ts	8	TokenManager 存取删
组件测试	TransactionCard.test.ts	13	金额符号、商户名、样式类
Mock API 集成	transactions.test.ts	12	成功/401/422/网络错误
Store 测试	user.test.ts	7	登录登出状态流转
合计	4 文件	40	全部通过

```
> personal-financial-management-assistant-frontend@1.0.0 test
> vitest run --coverage

RUN v2.1.9 C:/Users/22965/Desktop/移动计算/personal-financial-management-assistant/frontend
Coverage enabled with v8

✓ src/__tests__/api/transactions.test.ts (12)
✓ src/__tests__/components/TransactionCard.test.ts (13) 333ms
✓ src/__tests__/utils/auth.test.ts (8)
✓ src/__tests__/stores/user.test.ts (7)

Test Files  4 passed (4)
Tests       40 passed (40)
Start at    00:54:10
Duration    25.15s (transform 3.11s, setup 45.70s, collect 4.93s, tests 524ms, environment 34.67s, prepare 4.89s)

% Coverage report from v8
-----
File                                     % Stmts   % Branch   % Funcs   % Lines   Uncovered Line #s
-----
All files                               94.15     83.67     100      94.15
components/business                     95.12     82.35     100      95.12
  TransactionCard.vue                   95.12     82.35     100      95.12  68-71
stores                                  90         80     100       90
  user.ts                               90         80     100       90  37,53-57
utils                                  100        100     100      100
  auth.ts                               100        100     100      100
-----
(base) PS C:\Users\22965\Desktop\移动计算\personal-financial-management-assistant\frontend>
```

8.2 后端测试 (pytest)

类型与数量：

类型	代表文件	用例数	说明
单元测试	test_services.py	16	密码/JWT、AI 规则匹配、Mock LLM
单元测试	test_ai_budget.py	4	预算JSON 归一化
API 接口测试	test_accounts.py 等	74	TestClient + SQLite
合计	8 文件	94	CI 全部通过

```
(base) PS C:\Users\22965\Desktop\移动计算\personal-financial-management-assistant\backend> uv run pytest tests/ -v --cov
=app
===== test session starts =====
platform win32 -- Python 3.13.2, pytest-9.0.3, pluggy-1.6.0 -- C:\Users\22965\Desktop\移动计算\personal-financial-manage
ment-assistant\backend\.venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\22965\Desktop\移动计算\personal-financial-management-assistant\backend
configfile: pyproject.toml
plugins: anyio-4.12.1, asyncio-1.3.0, cov-7.1.0
asyncio: mode=Mode.AUTO, debug=False, asyncio_default_fixture_loop_scope=None, asyncio_default_test_loop_scope=function
collected 94 items

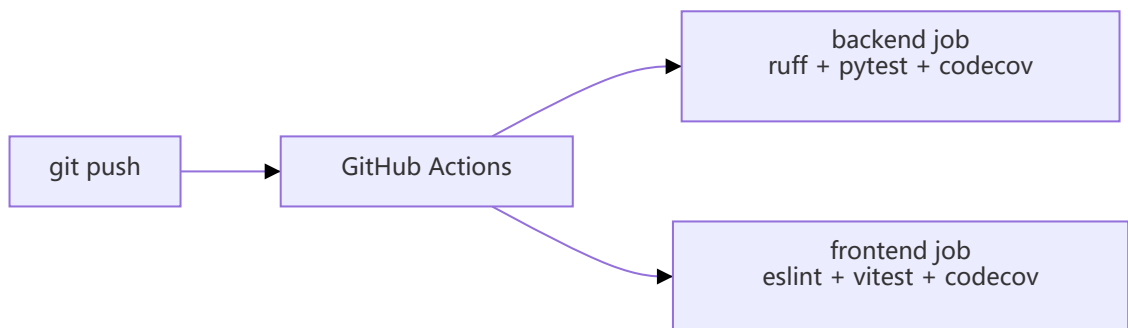
tests/test_accounts.py::TestCreateAccount::test_create_account PASSED [ 1%]
tests/test_accounts.py::TestCreateAccount::test_create_account_default PASSED [ 2%]
tests/test_accounts.py::TestCreateAccount::test_create_account_duplicate_name PASSED [ 3%]
tests/test_accounts.py::TestCreateAccount::test_create_account_unauthorized PASSED [ 4%]
tests/test_accounts.py::TestGetAccounts::test_get_accounts_list PASSED [ 5%]
tests/test_accounts.py::TestGetAccounts::test_get_accounts_filter_by_type PASSED [ 6%]
tests/test_accounts.py::TestGetAccounts::test_get_accounts_filter_enabled PASSED [ 7%]
tests/test_accounts.py::TestAccountSummary::test_get_account_summary PASSED [ 8%]
tests/test_accounts.py::TestAccountSummary::test_get_account_summary_multiple_accounts PASSED [ 9%]
tests/test_accounts.py::TestDefaultAccount::test_get_default_account PASSED [ 10%]
tests/test_accounts.py::TestDefaultAccount::test_get_default_account_no_default PASSED [ 11%]
tests/test_accounts.py::TestDefaultAccount::test_get_default_account_exists PASSED [ 12%]
tests/test_accounts.py::TestTransfer::test_transfer PASSED [ 13%]
tests/test_accounts.py::TestTransfer::test_transfer_insufficient_balance PASSED [ 14%]
tests/test_accounts.py::TestTransfer::test_transfer_same_account PASSED [ 15%]
tests/test_accounts.py::TestTransfer::test_transfer_account_not_found PASSED [ 17%]
tests/test_accounts.py::TestDeleteAccount::test_delete_account PASSED [ 18%]
tests/test_accounts.py::TestDeleteAccount::test_delete_account_with_transactions PASSED [ 19%]

tests/test_services.py::TestAIServiceRuleMatching::test_match_transport PASSED [ 93%]
tests/test_services.py::TestAIServiceRuleMatching::test_match_shopping PASSED [ 94%]
tests/test_services.py::TestAIServiceRuleMatching::test_match_medical PASSED [ 95%]
tests/test_services.py::TestAIServiceRuleMatching::test_no_match PASSED [ 96%]
tests/test_services.py::TestAIServiceRuleMatching::test_empty_merchant_name PASSED [ 97%]
tests/test_services.py::TestAIServiceRuleMatching::test_none_merchant_name PASSED [ 98%]
tests/test_services.py::TestAIServiceAdvice::test_generate_advice_no_transactions PASSED [100%]

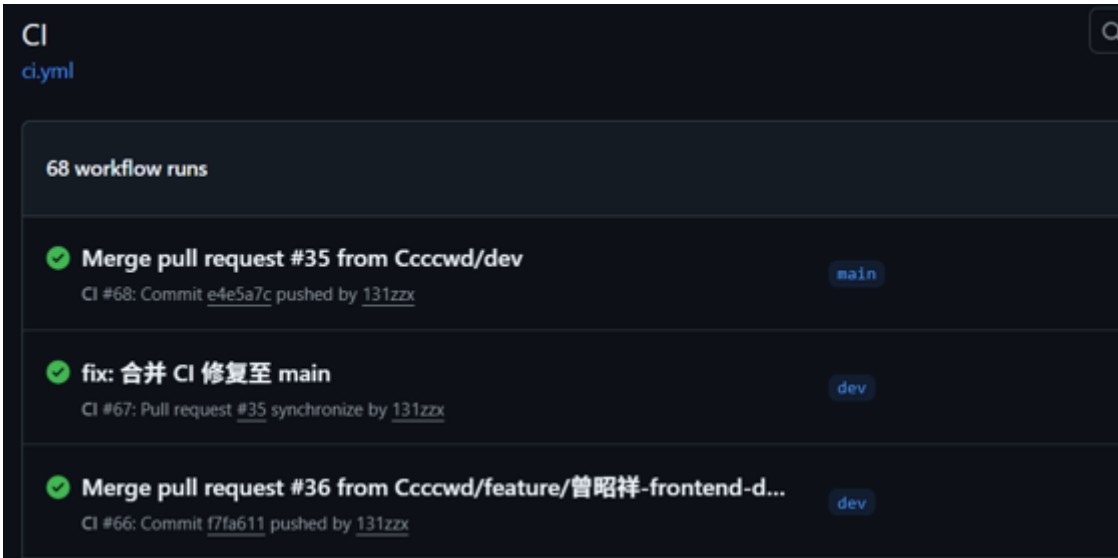
app\services\wechat_bill_service.py 271 240 11% 53-62, 66-68, 72-109, 115, 122-219, 230-292, 298-304, 308-31
2, 317-362, 376, 387-393, 397-402, 406-434, 448-461, 465-471, 475-488, 492-497
app\utils\logger.py 30 6 80% 16-36
-----
TOTAL 2865 1294 55%
===== 94 passed, 3152 warnings in 288.51s (0:04:48) =====
```

九、CI/CD 持续集成

`.github/workflows/ci.yml` 在 push 到 `main / dev` 及 PR 时触发，**前后端并行**：



Job	步骤
backend	uv sync → ruff check → pytest --cov → Codecov
frontend	npm install → eslint → vitest run --coverage → Codecov



十、安全审查与加固

本项目采用 **Vibe Coding + OWASP Top 10** 工作流，由陈伟栋主导完成 AI 辅助代码审查，审查记录见 `docs/security-review.md`。

10.1 审查范围与方法

审查覆盖后端核心路径：

模块	文件	关注点
认证	<code>app/core/security.py</code> 、 <code>app/api/auth.py</code>	密码哈希、JWT、黑名单
鉴权	<code>app/core/dependencies.py</code>	受保护路由、用户隔离
业务 API	<code>transactions.py</code> 、 <code>accounts.py</code> 等	越权、参数校验
AI 服务	<code>ai_service.py</code>	API Key 环境变量、Prompt 注入
入口	<code>main.py</code>	CORS、中间件、错误暴露

使用 AI 按 OWASP Top 10 视角审查后，记录问题与修复方案，并在代码中落地。

10.2 已修复的典型问题

问题	修复
<code>.env.example</code> 含真实密钥	改为占位符
无鉴权重置密码	移除裸 <code>/reset-password</code> ，改 Token 流程
Refresh Token 未查黑名单	<code>verify_refresh_token</code> 增加检查
debug 模式返回 SQL 细节	异常改 logging，响应泛化
<code>sort_by</code> 任意列	白名单四字段
无速率限制	<code>RateLimitMiddleware</code>

10.3 运行时安全加固

(1) 安全 HTTP 响应头 (`SecurityHeadersMiddleware`)：

- `X-Content-Type-Options: nosniff`
- `X-Frame-Options: DENY`
- `X-XSS-Protection: 1; mode=block`
- `Content-Security-Policy` 限制脚本与连接来源

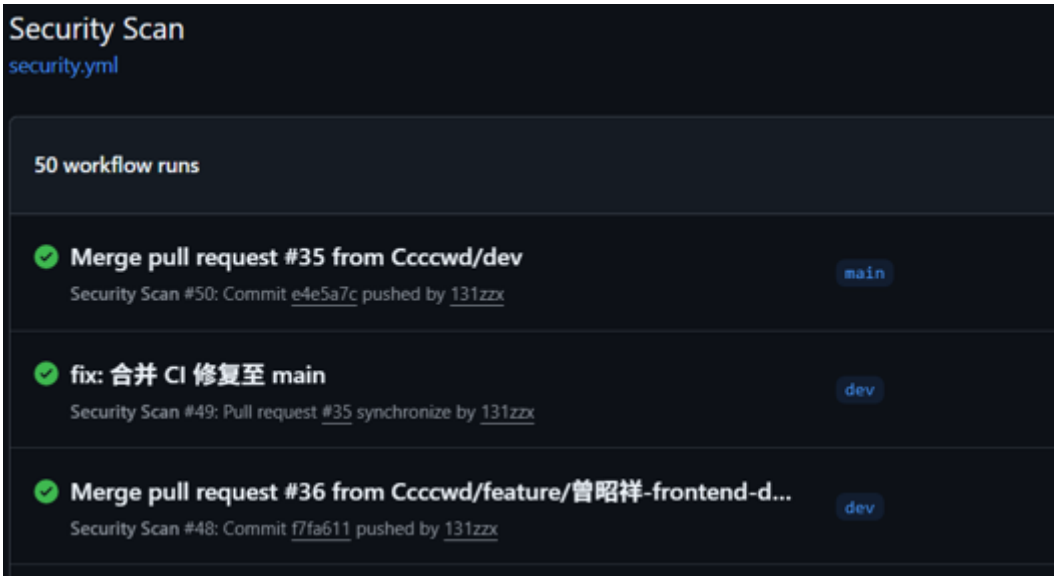
(2) 认证清单 (当前状态)

- ✓ BCrypt 密码存储
- ✓ JWT 过期 + 登出黑名单 (Redis)
- ✓ 业务接口 `get_current_active_user` 鉴权
- ✓ 全表 `user_id` 过滤防越权
- ✓ ORM 参数化查询，无 SQL 拼接
- ✓ `.env` 不入库，`.env.example` 无真实密钥

10.4 CI 自动化安全扫描

`.github/workflows/security.yml` 包含两个并行 Job：

Job	工具	作用
gitleaks	Gitleaks Action	扫描 Git 历史硬编码密钥
pip-audit	pip-audit	检测 Python 依赖已知 CVE



十一、Docker 容器化部署

Docker 部署目标：本地一条命令启动全栈，且生产镜像体积小、非 root 运行。

11.1 Compose 编排结构

根目录采用 **Compose V2** 多文件合并：

文件	作用
<code>compose.yml</code>	后端 + MySQL + Redis（开发热重载）
<code>compose.override.yml</code>	追加 frontend 生产镜像（本地默认合并）
<code>compose.prod.yml</code>	生产资源限制、Secrets、GHCR 镜像

一键启动：

```
docker compose up -d
```

服务	容器名	端口映射	健康检查
frontend	finance-frontend	3000→8080	wget <code>/health</code>
backend	finance-backend	8000→8000	<code>/api/health</code>
mysql	finance-mysql	3307→3306	mysqladmin ping
redis	finance-redis	6379→6379	redis-cli ping

MySQL/Redis 数据卷持久化（`mysql_data`、`redis_data`），重启容器不丢库。

11.2 前端 Dockerfile（三阶段）

阶段	基础镜像	用途
development	node:20-bookworm-slim	npm run dev，配合 volume 热重载
builder	node:20-bookworm-slim	npm run build，注入 VITE_API_BASE_URL=/api
production	nginxinc/nginx-unprivileged	托管 dist/，用户非 root，监听 8080

构建时注入 `VITE_API_BASE_URL=/api`，与 Nginx `location /api/` 及 Axios 默认 baseURL 一致，避免硬编码 Railway 地址。

11.3 后端 Dockerfile

- 基于 `python:3.12-slim` 多阶段构建，uv 安装依赖；
- 非 root 用户 `appuser` 运行 `uvicorn`；
- 内置 `HEALTHCHECK` 探测 `/api/health`。

11.4 Nginx 反向代理

`frontend/nginx/default.conf` 关键规则：

- `try_files` 支持 Vue History 路由；
- `location /api/` → `proxy_pass http://backend:8000/api/`；
- 独立 `/health` 供容器健康检查。

浏览器始终访问 `http://localhost:3000`，API 走同源 `/api`，无 CORS 预检问题。

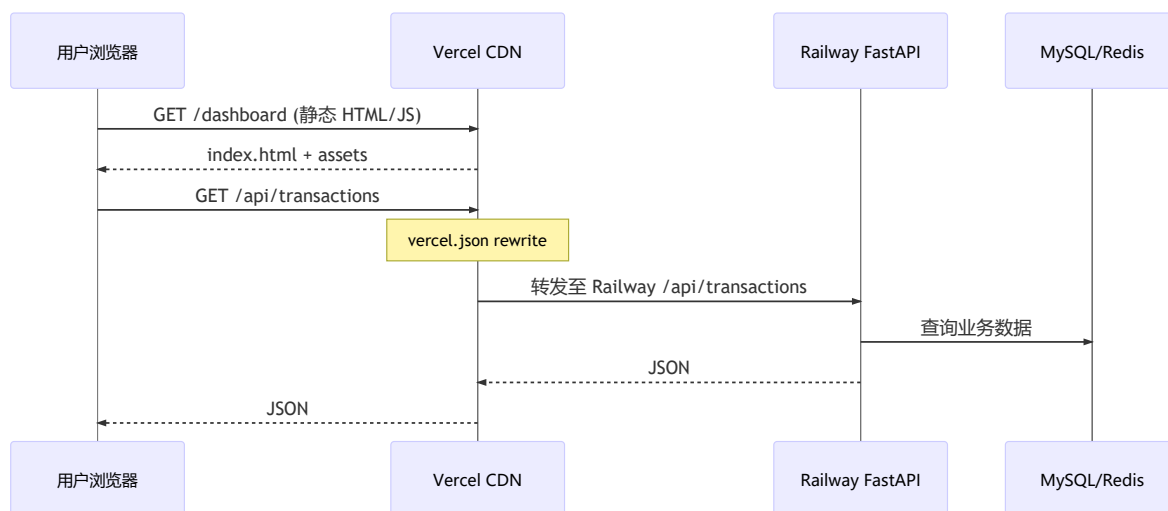
11.5 部署效果

```
(base) PS C:\Users\22965\Desktop\移动计算\personal-financial-management-assistant> docker compose ps
time="2026-06-17T01:49:53+08:00" level=warning msg="Found multiple config files with supported names: C:\\Users\\22965\\Desktop\\移动计算\\personal-financia
l-management-assistant\\compose.yaml, C:\\Users\\22965\\Desktop\\移动计算\\personal-financial-management-assistant\\docker-compose.yml"
time="2026-06-17T01:49:53+08:00" level=warning msg="Using C:\\Users\\22965\\Desktop\\移动计算\\personal-financial-management-assistant\\compose.yaml"
NAME                IMAGE                                COMMAND                  SERVICE    CREATED   STATUS    PORTS
finance-backend     personal-financial-management-assistant-backend  "uvicorn main:app --..." backend     7 hours ago Up 3 hours (healthy) 0.0.0.0:8000-
>8000/tcp, [::]:8000->8000/tcp
finance-frontend    personal-financial-management-assistant-frontend  "/docker-entrypoint.sh..." frontend    7 hours ago Up 3 hours (healthy) 0.0.0.0:3000-
>8000/tcp, [::]:3000->8000/tcp
finance-mysql       mysql:8.0                                "docker-entrypoint.sh..." mysql       7 hours ago Up 5 hours (healthy) 0.0.0.0:3307-
>3306/tcp, [::]:3307->3306/tcp
finance-redis       redis:7-alpine                          "docker-entrypoint.sh..." redis       7 hours ago Up 5 hours (healthy) 0.0.0.0:6379-
>6379/tcp, [::]:6379->6379/tcp
(base) PS C:\Users\22965\Desktop\移动计算\personal-financial-management-assistant> docker compose up -d
time="2026-06-27T17:38:13+08:00" level=warning msg="Found multiple config files with supported names: C:\\Users\\22965\\
Desktop\\移动计算\\personal-financial-management-assistant\\compose.yaml, C:\\Users\\22965\\Desktop\\移动计算\\personal-
financial-management-assistant\\docker-compose.yml"
time="2026-06-27T17:38:13+08:00" level=warning msg="Using C:\\Users\\22965\\Desktop\\移动计算\\personal-financial-manage
ment-assistant\\compose.yaml"
[+] Running 4/4
✔ Container finance-mysql      Healthy      0.6s
✔ Container finance-redis      Healthy      0.6s
✔ Container finance-backend    Running      0.0s
✔ Container finance-frontend    Running      0.0s
```

十二、云服务部署

线上采用**前后端分离双平台**：前端 Vercel 托管静态资源，后端 Railway 运行 FastAPI + MySQL + Redis。推送 `main` 分支可触发自动部署。前端部署使用的仓库是 fork 的仓库（账号问题，仓库不是创建在前端部署的github账号下，只是合作开发无法直接使用仓库部署到Vercel），每次更新后推送到该 fork 仓库的 `main` 分支即可，同样触发自动部署。

12.1 整体流量路径



12.2 Vercel 前端部署

项目配置 `vercel.json`:

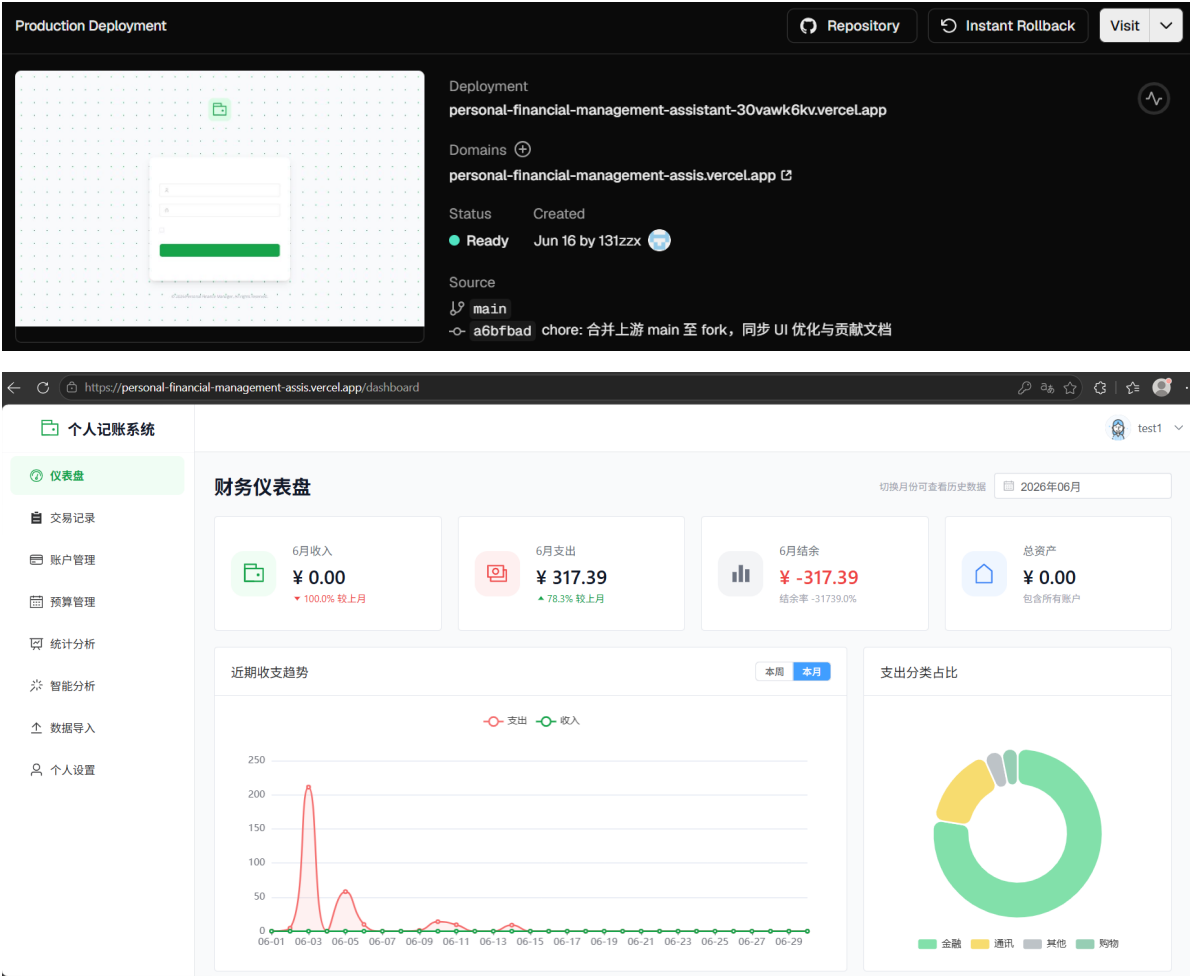
```
{
  "installCommand": "cd frontend && npm install",
  "buildCommand": "cd frontend && npm run build",
  "outputDirectory": "frontend/dist",
  "rewrites": [
    {
      "source": "/api/:path*",
      "destination": "https://personal-financial-management-assistant-production.up.railway.app/api/:path*"
    },
    {
      "source": "/(.*)",
      "destination": "/index.html"
    }
  ]
}
```

要点:

1. **构建**: 在 `frontend/` 下执行 `npm run build`, 产物为 `frontend/dist`。
2. **SPA 路由**: 除 `/api` 外所有路径 rewrite 到 `index.html`, 避免刷新 404。
3. **API 反代**: 浏览器仍请求同源 `/api/*`, 由 Vercel 转发 Railway, 前端无需写死后端域名。

4. 环境变量：构建时可设 `VITE_API_BASE_URL=/api`（与本地 Docker 策略一致）。

线上地址：<https://personal-financial-management-assis.vercel.app>



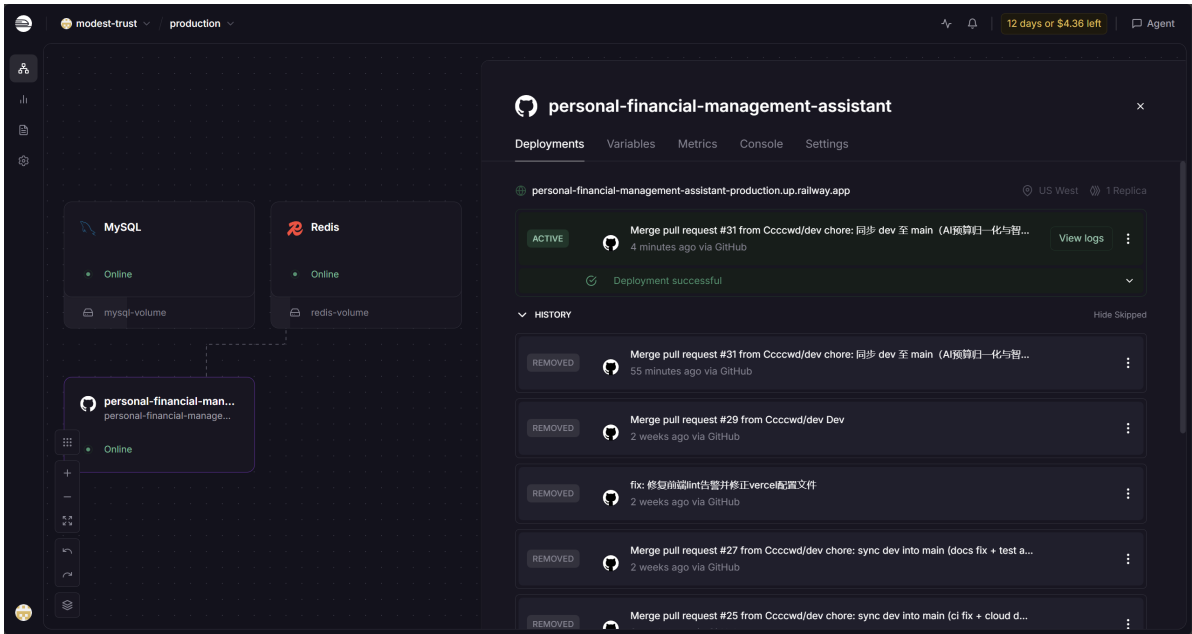
12.3 Railway 后端部署

配置项	值
Root Directory	backend
构建	backend/Dockerfile + railway.toml
数据库	Railway 插件 MySQL、Redis
触发	Git push main 自动 redeploy

关键环境变量（Variables 面板配置，密钥不提交 Git）：

```
DATABASE_URL=mysql+pymysql://${MySQL.MYSQL_USER}:...@...
REDIS_URL=${Redis.REDIS_URL}
SECRET_KEY=<随机强密钥>
AI_API_KEY=<DeepSeek Key>
AI_BASE_URL=https://api.deepseek.com
AI_MODEL=deepseek-chat
APP_ENV=production
DEBUG=false
```

Railway 通过 `${MySQL.*}` 变量引用自动注入数据库连接串，避免手写密码。



十三、监控与可观测性

后端实现三项能力：

能力	端点/模块	内容
结构化日志	app/utils/logger.py	生产 JSON，开发纯文本
健康检查	GET /api/health	MySQL/Redis/uptime
指标	GET /api/metrics	请求数、错误率、均耗时、Top 路径

MetricsMiddleware 在 main.py 注册，自动采集每个请求。



十四、功能展示

演示视频在/docs目录下，裁剪掉了部分较长等待或者操作错误的片段。

14.1 认证与仪表盘

登录页与注册页：

个人记账系统

智能财务助手，轻松管理您的每一笔收支

欢迎登录

test4

☐ 记住我

忘记密码?

登录

还没有账号? 立即注册

个人记账系统

开启您的智能财务管理之旅

用户注册

请输入用户名

请输入邮箱

请输入密码

请再次输入密码

注册

已有账号? 立即登录

© 2026 Personal Finance Manager. All rights reserved.

仪表盘：

个人记账系统

仪表盘

交易记录

账户管理

预算管理

统计分析

智能分析

数据导入

个人设置

财务仪表盘

切换月份可查看历史数据 2026年06月

6月收入
¥ 1100.00
▼ 26.7% 较上月

6月支出
¥ 1264.39
▲ 32.8% 较上月

6月结余
¥ -164.39
结余率 -14.9%

总资产
¥ 1753.00
包含所有账户

近期收支趋势

本月 上月

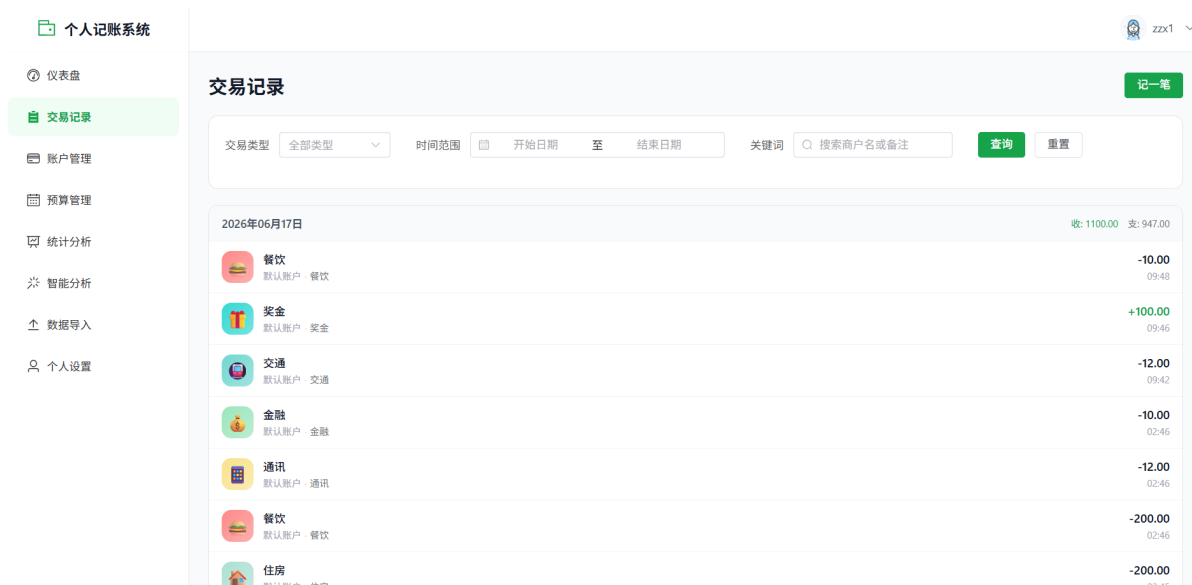
支出 收入

支出分类占比



14.2 记账与账户

交易记录：



记录详情：



手动记账：

个人记账系统

仪表盘

交易记录

账户管理

预算管理

统计分析

智能分析

数据导入

个人设置

zzx1

记一笔

支出

收入

转账

金额：¥100

选择分类

餐饮

交通

购物

娱乐

医疗

教育

住房

通讯

金融

其他

账户

保存支出

账户管理：

个人记账系统

仪表盘

交易记录

账户管理

预算管理

统计分析

智能分析

数据导入

个人设置

zzx1

账户管理

+新增账户

总资产

1753.00

账户总数

3 个

默认账户

现金

¥153.00

转账 编辑 删除

1

银行卡

¥100.00

转账 编辑 删除

22

支付宝

¥1500.00

转账 编辑 删除

个人记账系统

仪表盘

交易记录

账户管理

预算管理

统计分析

智能分析

数据导入

个人设置

zzx1

返回账户

编辑

与 转账

调整余额

删除

默认账户

现金

默认

当前余额

¥153.00

累计收入 ¥5,905.20 累计支出 ¥6,405.20

交易记录

全部类型

2026-06-17

餐饮

餐饮 09:48

¥100.00

奖金

奖金 09:46

¥100.00

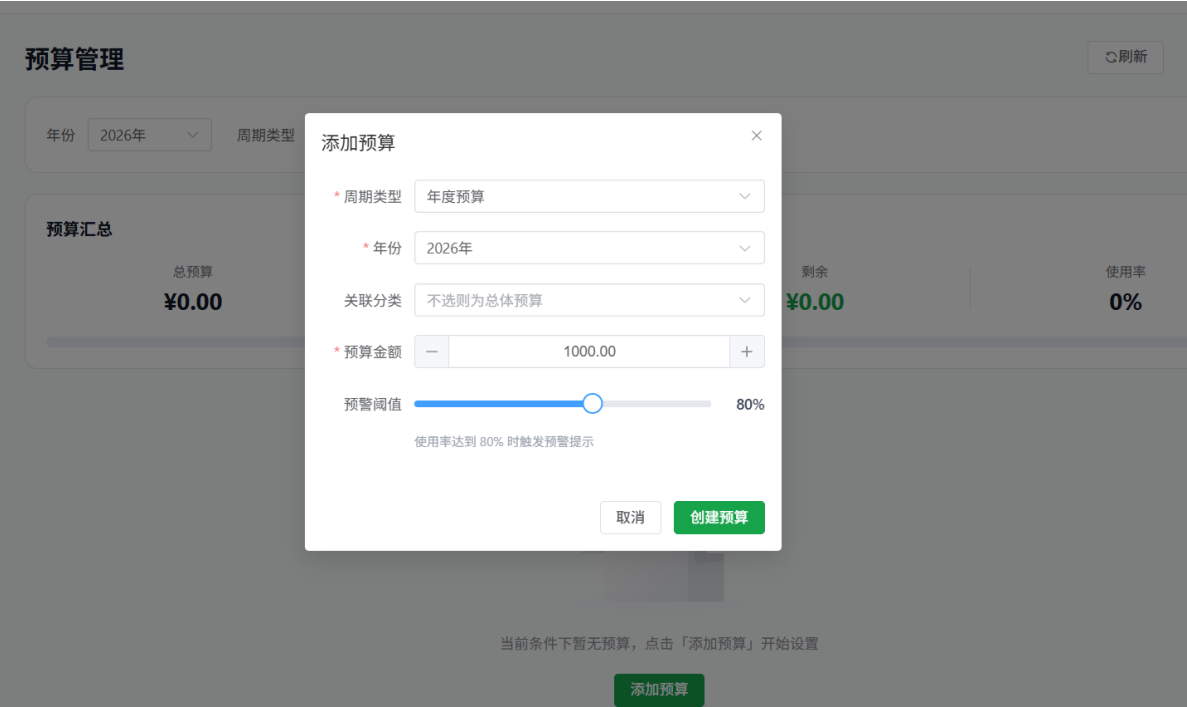
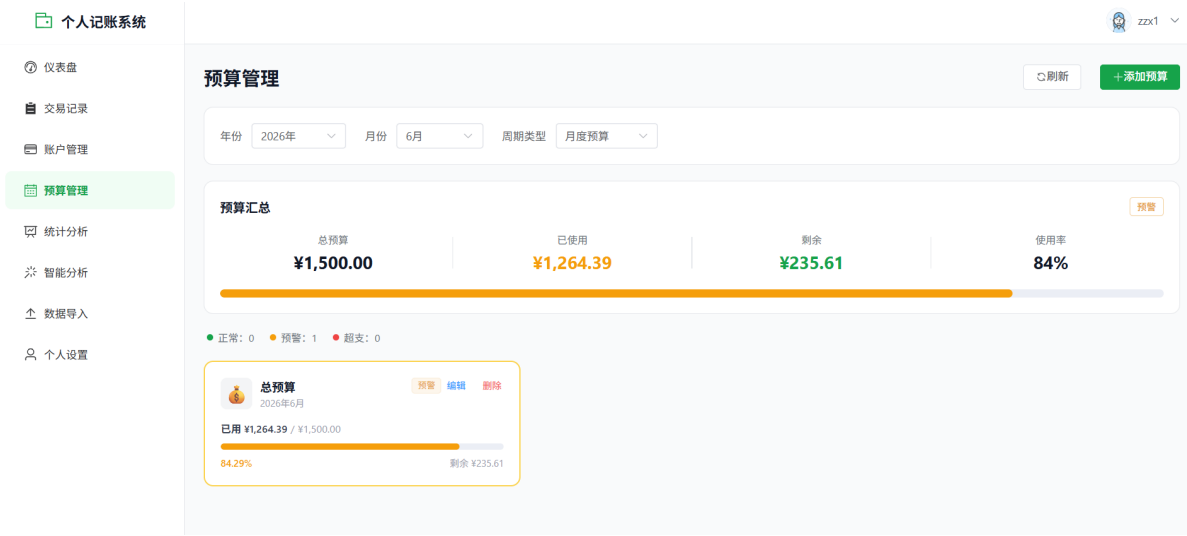
交通

交通 09:42

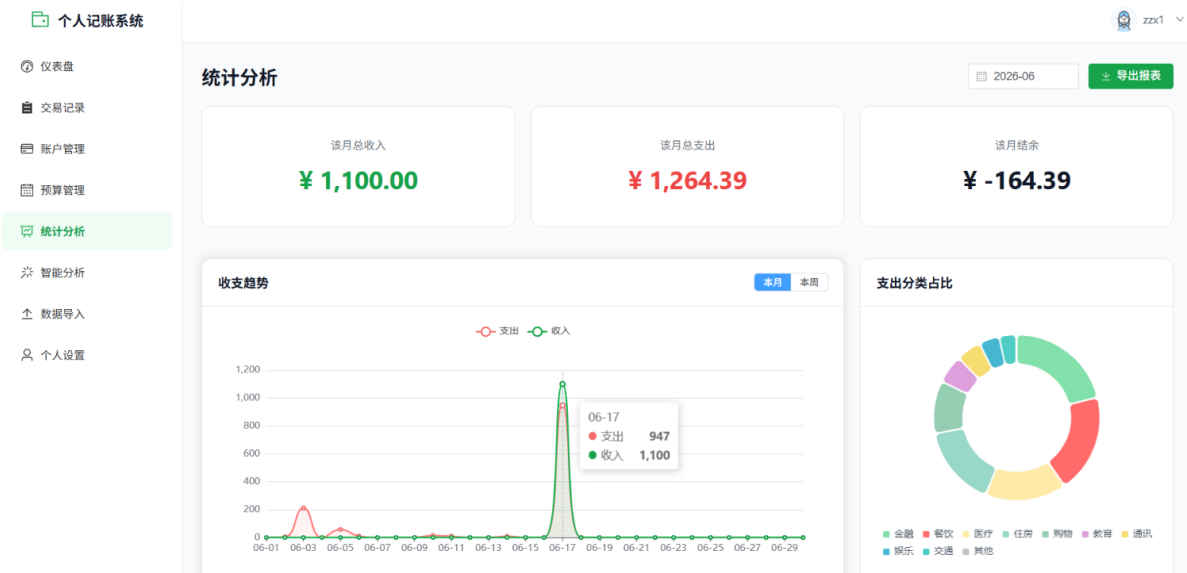
¥12.00

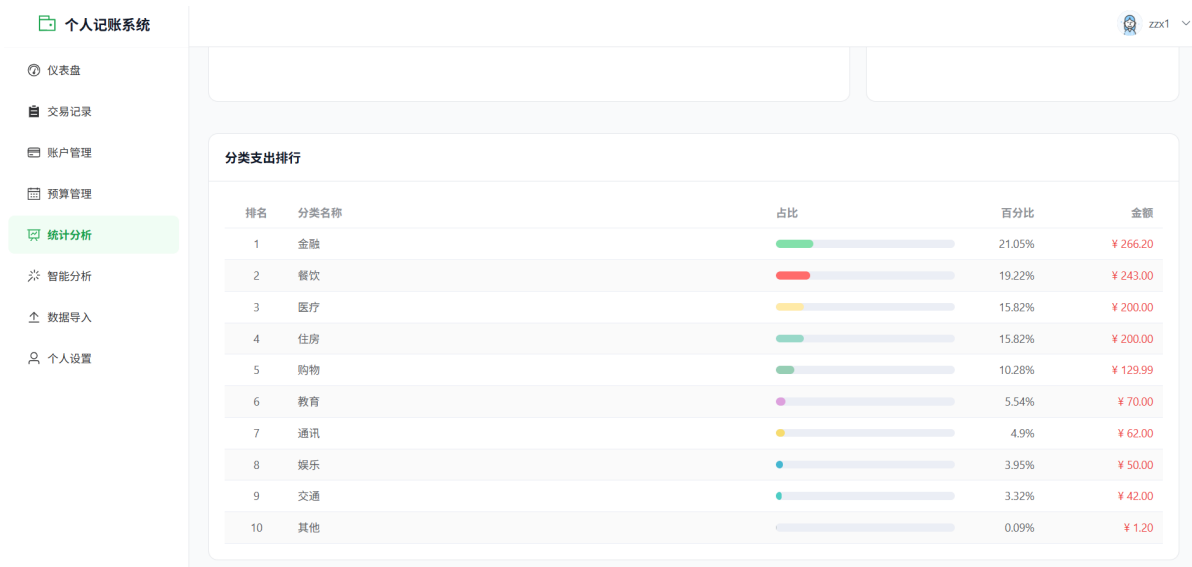
14.3 预算与统计

预算管理：



统计分析：





14.4 导入与 AI

微信账单一键导入：

个人记账系统

仪表盘

交易记录

账户管理

预算管理

统计分析

智能分析

数据导入

个人设置

微信账单导入

支持微信支付导出的 CSV / XLSX 格式账单文件

导入历史 导出说明

1 上传文件

2 预览确认

3 导入结果

拖拽账单文件到此处，或 点击选择

支持微信支付导出的 .csv 和 .xlsx 文件，最大 10 MB

如何导出微信账单？

1 打开微信 → 我 → 服务 → 钱包

2 点击右上角 账单 → 常见问题 → 下载账单

3 选择时间范围后发送至邮箱，解压缩包后上传 CSV 或 XLSX 文件

个人记账系统

仪表盘

交易记录

账户管理

预算管理

统计分析

智能分析

数据导入

个人设置

1 上传文件

2 预览确认

3 导入结果

36 总笔数

1 收入笔数

35 支出笔数

账单周期: 2026-05-17 至 2026-06-14

交易时间	交易对方	商品说明	收/支	金额 (元)	支付方式
2026-06-14 11:03	中融信收款	先购后付	支出	-9.00	邮储银行储蓄卡[...]
2026-06-11 12:44	中融信收款	先购后付	支出	-9.50	邮储银行储蓄卡[...]
2026-06-10 19:29	中融信收款	先购后付	支出	-4.00	零钱
2026-06-10 11:14	杭州深度求索	DeepSeek-API服务(229****2...	支出	-10.00	零钱
2026-06-09 12:01	l_l_l_l_l_l_l	/	支出	-1.20	零钱
2026-06-06 10:25	东莞市道滘镇冬百...	订单支付: 202606061024580...	支出	-9.99	零钱
2026-06-05 23:05	中国移动	为198****2016话费充值	支出	-50.00	零钱
2026-06-05 18:09	中融信	先购后付	支出	-8.00	零钱
2026-06-03 19:11	中融信	先购后付	支出	-11.00	零钱
2026-06-03 12:18	曾昭祥	转账到银行卡	支出	-200.20	零钱

导入设置

导入到账户

默认账户 (现金)

不选则自动使用默认账户

默认分类

不指定 (可后续手动分类)

所有导入记录将使用该分类，可留空后续手动修改

系统自动识别并跳过已导入的重复交易

重新上传 确认导入

个人记账系统

仪表盘

交易记录

账户管理

预算管理

统计分析

智能分析

数据导入

个人设置

微信账单导入

支持微信支付导出的 CSV / XLSX 格式账单文件

导入历史

导出说明

上传文件

预览确认

导入结果

导入完成

36 共处理

0 成功入账

36 跳过重复

继续导入

查看账单流水

AI理财建议：

个人记账系统

仪表盘

交易记录

账户管理

预算管理

统计分析

智能分析

数据导入

个人设置

智能分析

近 3 个月

生成分析

智能建议

历史记录

分析周期：2026-03-29 – 2026-06-27 生成时间：2026-06-27 10:43:21 新生成

消费亮点

2条

医疗和住房支出控制得当，分别仅占6.4%，说明您对基本生活保障有合理规划。

餐饮支出占比7.7%，相对较低，显示您有良好的饮食习惯或有效控制了外出就餐开支。

优化建议

3条

审视金融支出明细，区分投资与费用，确保金融活动符合长期财务目标。

制定月度预算，将总支出控制在3000元左右，优先保障住房、医疗、餐饮等必要开支。

考虑增加教育投入至总预算的10%，用于技能提升或兴趣培养。

风险提示

2条

金融类支出占比高达64.2%，需确认是否包含投资或储蓄，若为费用支出则比例过高，可能影响现金流。

教育支出仅占4.4%，若为终身学习投入，建议适度增加以提升个人价值。

下月预算建议总计：¥3000

个人记账系统

仪表盘

交易记录

账户管理

预算管理

统计分析

智能分析

数据导入

个人设置

智能分析

查看完整分析报告

金融

¥1500

餐饮

¥300

医疗

¥200

住房

¥200

教育

¥300

购物

¥150

通讯

¥100

娱乐

¥100

交通

¥100

其他

¥50

查看完整分析报告

您好！根据您近3个月的消费数据，您的总支出为3144.98元，共62笔交易。整体来看，您在医疗和住房方面控制得当，餐饮支出也较为合理，值得肯定。但金融类支出占比高达64.2%，需进一步分析其构成——如果是投资或储蓄，则是良好习惯；如果是手续费或利息支出，则建议优化。此外，教育支出偏低，建议适当增加自我投资。下月预算建议控制在3000元左右，重点保障住房、医疗和餐饮，并提高教育预算至300元。请定期审视金融支出，确保其与您的财务目标一致。祝您理财顺利！

个人记账系统

仪表盘

交易记录

账户管理

预算管理

统计分析

智能分析

数据导入

个人设置

zzx1

智能分析

近3个月

生成分析

智能建议

历史记录

生成时间	分析周期	摘要	操作
2026-06-27	2026-03-29 ~ 2026-06-27	医疗和住房支出控制得当，分别仅占6.4%，说明您对基本生活保障有合理规划。	查看
2026-06-27	2026-03-29 ~ 2026-06-27	总支出控制在合理范围内，月均约1048元，较为节俭。	查看
2026-06-17	2026-03-19 ~ 2026-06-17	总支出控制得当，三个月仅2197.98元，月均约732元，显示出良好的消费节制力。	查看
2026-06-17	2026-03-19 ~ 2026-06-17	总支出控制得当，三个月仅2197.98元，平均每月约733元，消费较为节制。	查看
2026-06-17	2026-03-19 ~ 2026-06-17	总支出控制在较低水平，仅为2197.98元，说明消费较为节制。	查看
2026-06-17	2026-03-19 ~ 2026-06-17	总支出控制在较低水平，整体消费较为节制。	查看
2026-06-17	2026-03-19 ~ 2026-06-17	总支出控制得当，三个月仅2197.98元，月均约732元，消费较为节制。	查看
2026-06-16	2026-03-18 ~ 2026-06-16	总支出控制在较低水平，仅1780.23元，显示良好的消费节制力	查看
2026-06-16	2026-03-18 ~ 2026-06-16	教育投资占比合理，体现了对自我提升的重视	查看
2026-06-16	2026-03-18 ~ 2026-06-16	总支出控制得当，三个月仅1780.23元，月均约593元，体现了良好的节俭习惯。	查看

共 13 条

10条/页

<12>

14.5 账号

个人账号管理：

zzx1

个人设置

基本信息

上传新头像

* 用户名

zzx1

注册邮箱

1122@qq.com

已验证

保存基本信息

密码与安全

* 当前密码

请输入当前密码

* 新的密码

请输入新密码（不少于6位）

* 确认密码

请再次输入新密码

更新账号密码

十五、总结与展望

15.1 项目总结

本项目完成了从需求分析、原型设计、前后端分离开发、AI 集成、自动化测试、CI/CD、安全审查到 Docker/云部署的完整 Web 应用工程实践。

15.2 问题与反思

前后端联调与类型约定

- MySQL `DECIMAL` 经 FastAPI 序列化后常为字符串，前端若直接调用 `.toFixed()` 会导致账户/预算页白屏；应在展示层统一 `Number()` 转换或封装格式化函数，并在 API 中明确金额字段类型。
- JWT 的 `sub` 必须为字符串；若后端误传整数，会出现「登录成功但 `/auth/me` 持续 401」的隐蔽问题。
- Token 过期或无效时须由 Axios 响应拦截器清除 `localStorage` 并跳转登录，否则页面停留在半登录态。
- Windows 开发环境下 Vite 代理 `localhost:8000` 可能解析到 IPv6 `:::1`，而后端绑定 `127.0.0.1`，导致代理 `ECONNREFUSED`；统一改为 `127.0.0.1` 可避免。
- 前后端模块导出名不一致（如 `getUserInfo` vs `getCurrentUser`）会引发 ES Module 加载失败、整页空白，联调时应先对齐 `import/export` 命名。

工程与部署

- 本地 dev、Docker Nginx、Vercel rewrite 三套环境均依赖「浏览器侧 `/api` 同源访问」策略，切换环境时只需改代理/rewrite 配置，业务代码不应硬编码后端域名。
- 前端 Vercel 部署因 GitHub 账号归属问题需使用 fork 仓库，与后端 Railway 直连主仓库形成「双仓库、双平台」运维路径，合并代码后需分别推送到对应仓库的 `main` 分支才能触发两侧部署。
- Docker 四服务联调时 MySQL 健康检查、`backend` 依赖顺序与 Nginx `/api` 反代规则需一并验证，否则前端页面可访问但 API 全部失败。

安全与可观测性

- 安全审查曾发现 CORS 过宽、部分接口缺少速率限制等问题，已在代码中修复。
- 监控能力目前仅提供后端 `/api/health` 与 `/api/metrics`，未建设前端可视化监控页面，运维排查主要依赖 Railway 日志与手动 curl 探测。

15.3 优化方向

- web端对随时记账需求不友好，开发移动端适配优化；
- Playwright E2E 覆盖核心记账链路；
- 多账单源导入（支付宝等）；
- 提供前端监控页面；
- AI 建议与预算模块联动（如一键采纳 AI 预算）。

参考文献

- 课程课件
- [Vue 3 官方文档](#)
- [FastAPI 官方文档](#)
- [DeepSeek API 文档](#)
- [Element Plus 组件库](#)
- 项目仓库：<https://github.com/Ccccwd/personal-financial-management-assistant>

